UNITÀ 5 · SITI WEB RESPONSIVE E ACCESSIBILI · LEZIONE 8

Responsive senza media query

E se fosse il layout a capire da solo quanto spazio ha? clamp(), auto-fit, minmax, flex-wrap e le container query: poche righe al posto di tanti breakpoint.

PER COMINCIARE

Quante media query servono per fare un sito responsive?

SONDAGGIO

- **A** Almeno 3: telefono, tablet, PC
- **B** Una o due
- **C** Dipende dal sito
- **D** Anche zero

MENTI CHE CI HANNO PRECEDUTO

Il web non è una pagina stampata

> **Dovremmo accettare il fatto che il web non ha gli stessi vincoli della carta, e progettare per questa flessibilità.**
>
> **John Allsopp**, sviluppatore australiano, "A Dao of Web Design", A List Apart, 2000

Nel 2018 Jen Simmons lo chiama intrinsic web design: il layout dipende dal contenuto e dallo spazio disponibile, non da un elenco di dispositivi.

LE MISURE FLUIDE

Unità che si adattano da sole

Unità	Rispetto a che cosa	Esempio su un telefono largo 390 px
`%`	il contenitore	`width: 50%` in un box di 300 px = 150 px
`vw`	1% della larghezza del viewport	`50vw` = 195 px
`rem`	la dimensione del testo della pagina (di solito 16 px)	`1.5rem` = 24 px
`ch`	la larghezza del carattere "0" del font	`max-width: 65ch` = righe comode da leggere
`fr`	una parte dello spazio libero in una griglia	`1fr 2fr` = un terzo e due terzi

MIN(), MAX(), CLAMP()

Un valore che cresce, ma con un pavimento e un soffitto

clamp(minimo, preferito, massimo)

```
font-size: clamp(1.5rem, 1rem + 4vw, 3rem);
```

Il browser calcola il valore preferito; se scende sotto il minimo usa il minimo, se supera il massimo usa il massimo.

Viewport	1rem + 4vw	Risultato
300 px	16 + 12 = 28	28 px
400 px	16 + 16 = 32	32 px
800 px	16 + 32 = 48	48 px
1200 px	16 + 48 = 64	48 px

`width: min(100% - 32px, 70ch)`: larga quanto lo schermo meno i margini, ma mai più di 70 caratteri.

METTITI ALLA PROVA

Viewport largo 1000 px: quanto è grande il titolo?

```
h1 { font-size: clamp(1.5rem, 1rem + 4vw, 3rem); }   /* 1rem = 16px */
```

QUIZ

- **A** 56 px
- **B** 40 px
- **C** 24 px
- **D** 48 px

AL COMPUTER · 15 minuti

Testo fluido, misurato col righello

1. Crea **fluido.html** con meta viewport, un `<main class="pagina">`, un `<h1>` e tre paragrafi lunghi.
2. Nello stile: h1 con `font-size: clamp(1.5rem, 1rem + 4vw, 3rem)`.
3. `.pagina` con `width: min(100% - 32px, 70ch)` e `margin-inline: auto`.
4. In Chrome, modalità dispositivo a **400** px: in **Elementi** seleziona l'h1 e leggi *font-size* nella scheda **Calcolato**.
5. Ripeti a 800 e a 1200 px. Annota i tre valori.

Controllo: 32 px a 400, 48 px a 800, ancora 48 px a 1200. Il testo dei paragrafi non supera mai circa 70 caratteri per riga, e a 400 px ha 16 px di margine per lato.

FLEXBOX CHE VA A CAPO

Ogni elemento chiede un minimo, poi cresce

```
.schede {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}
.scheda {
  flex: 1 1 250px;
}
```

- `flex-basis: 250px`: ogni scheda parte da 250 px
- se non ci stanno, la riga va a capo
- `flex-grow: 1`: lo spazio che avanza si divide
- con 4 schede, a 600 px: 2 per riga, larghe 292 px

GRID CHE CONTA DA SOLA LE COLONNE

Una riga per una galleria responsive

```
.galleria { display: grid; gap: 20px;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr)); }
```

repeat(auto-fit, …)

Il browser crea **tutte le colonne che ci stanno**.

minmax(240px, 1fr)

Ogni colonna è larga almeno 240 px; lo spazio in più si divide.

Contenitore 1000 px

4 colonne servirebbero 4 × 240 + 3 × 20 = 1020 px: troppo. Ne stanno **3**, larghe (1000 − 40) ÷ 3 = 320 px.

QUIZ A TRABOCCHETTO

auto-fill con solo due card: quanto sono larghe?

```
.galleria {
  width: 1000px; display: grid; gap: 0;
  grid-template-columns:
    repeat(auto-fill, minmax(200px, 1fr));
}  /* dentro ci sono 2 card */
```

Con `auto-fit` la risposta sarebbe diversa.

QUIZ

- **A** 200 px
- **B** 500 px
- **C** 1000 px
- **D** 250 px

AL COMPUTER · 20 minuti

La galleria senza media query, in CodePen

1. Su **codepen.io/pen** scrivi un `<ul class="tag">` con 8 parole e un `<div class="galleria">` con 9 `<article>`.
2. CSS: `.tag` con `display: flex; flex-wrap: wrap;` e un gap.
3. CSS: `.galleria` con la griglia RAM vista prima, usando `minmax(min(240px, 100%), 1fr)`.
4. Trascina il bordo dell'anteprima. Poi passa ad `auto-fill` con solo 2 article.

Controllo: trascinando, la galleria passa da 1 a 2, 3, 4 colonne e i tag vanno a capo, senza nessuna `@media` nel codice. Con 2 article, `auto-fill` le lascia strette e `auto-fit` le allarga.

PUZZLE

Abbina la tecnica al risultato

`clamp(1rem, 2vw, 2rem)`	---	Prende il più piccolo dei due valori
`min(100%, 70ch)`	---	Rende un elemento interrogabile dalle container query
`flex-wrap: wrap`	---	Un valore fluido con un minimo e un massimo
`repeat(auto-fit, minmax(…))`	---	Tante colonne quante ne stanno
`container-type: inline-size`	---	Gli elementi vanno a capo se non c'è posto

CONTAINER QUERY

Il componente guarda il suo contenitore, non lo schermo

```
.contenitore {
  container-type: inline-size;
}
@container (min-width: 400px) {
  .card {
    display: grid;
    grid-template-columns: 120px 1fr;
  }
}
```



AL COMPUTER · 15 minuti

La stessa card in due contenitori

1. In un nuovo pen scrivi un `<main class="contenitore">` e un `<aside class="contenitore">`, affiancati con Flexbox.
2. Dai all'aside `flex: 0 0 280px` e al main `flex: 1`.
3. In entrambi metti la stessa `<article class="card">`: un riquadro colorato 120 × 120 e un paragrafo.
4. Copia il CSS della slide precedente e allarga l'anteprima oltre i 700 px.

Controllo: nel main la card è orizzontale (riquadro a sinistra, testo a destra); nell'aside, largo 280 px, la stessa card è verticale. Il codice della card è uno solo.

VIAGGIO NEL TEMPO

Come si impaginava il web: dalla più antica

1. Griglia fluida e media query: nasce il responsive
2. Flexbox supportato da tutti i browser principali
3. Colonne con float
4. Tabelle HTML usate per impaginare
5. CSS Grid arriva in tutti i browser principali
6. Container query in tutti i browser principali

QUANDO SERVONO ANCORA

Le media query non spariscono, cambiano mestiere

Ancora utili per

- cambi di struttura grandi (menu a scomparsa)
- preferenze: tema scuro, meno animazioni
- stampa: `@media print`



SFIDA FINALE

Apri il lucchetto della galleria



Contenitore largo **1000 px**, **gap 20 px**, colonne **repeat(auto-fit, minmax(240px, 1fr))**.

- 1ª cifra: quante colonne crea il browser
- ultime tre cifre: quanti pixel è larga ogni colonna

codice

BIGLIETTO D'USCITA

Completa prima di uscire

Su un viewport di 600 px, `10vw` vale ____ px.

`clamp(16px, 5vw, 24px)` a 1000 px vale ____ px.

Per avere tante colonne quante ne stanno uso `repeat(` _____ `, minmax(…))`.

Le query che guardano il contenitore si chiamano _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- Unità fluide: %, vw, rem, ch, fr
- `clamp(min, preferito, max)`, con una parte in rem
- `flex-wrap` + `flex-basis` per andare a capo
- `repeat(auto-fit, minmax())`: colonne automatiche
- auto-fill tiene le colonne vuote, auto-fit no
- Container query: il componente guarda il contenitore

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.